

안정적인 서비스를 만드는 클라우드 엔지니어 봉하석

CONTACT

PHONE : 010 5628 7676 | MAIL : HASEOK612@NAVER.COM

HYPERLINK

[HTTPS://GITHUB.COM/HASEOLC?TAB=REPOSITORIES](https://github.com/HASEOLC?tab=repositories)

CONTENTS

RESUME

- Building blocks
- 스킬 및 역량

01

PROJECT

- AWS 클라우드 인프라 자동화 및 보안 · 백업 환경 구축
- 하이브리드 클라우드 기반 중앙 집중형 에러 복구 파이프라인
- AWS 단일 클라우드 기반 AIOps · FinOps · DR 통합 운영 플랫폼

02

CORE COMPETENCIES

03

입사 후 포부

04

BUILDING BLOCKS

PR

자동화된 배포와 빠른 장애 분석이 가능한 클라우드 환경을 설계합니다.
AWS·Kubernetes 기반 인프라에 CI/CD, 모니터링, 보안을 연결해,
구축 이후의 운영까지 고려하는 클라우드 엔지니어입니다.

학력사항

2023.03 ~ 2025.02	국립 부경대학교 전자공학과 졸업
2015.03 ~ 2018.02	강릉 명륜고등학교 졸업

활동사항

2025.12 ~ 2026.07	Kt cloud 인프라 교육
2023.07 ~ 2024.08	SoC 학부연구생

자격사항

2026.06	정보처리기사
2025.10	리눅스마스터 2급
2025.09	네트워크관리사 2급

스킬 및 역량

CLOUD · NETWORK

AWS (VPC · EC2 · ALB · RDS)		VPC · EC2 · RDS · S3 기반 인프라 구성 및 IAM · KMS · Secrets 적용
Network · Security (SG · WAF)		Subnet · Route · SG · WAF 구성과 접근 제어 · 보안 정책 적용

IaC · CONTAINER

Terraform · IaC		HCL 기반 AWS IaC 구축 및 S3 Remote State를 통한 상태 관리
Docker · Kubernetes		Docker 이미지 배포 및 K3s 기반 Kubernetes 클러스터 구성

CI/CD · MONITORING

GitHub Actions		Push 기반 Terraform · Ansible · Kubernetes 자동 배포 파이프라인 구축
Prometheus · Grafana		서버 · Kubernetes 메트릭 수집 및 CPU · Memory · Pod 상태 시각화

PROJECT 1

01

AWS 클라우드 인프라 자동화 및 보안 · 백업 환경 구축



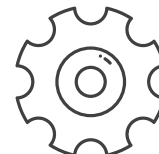
BACKGROUND

수작업 구축으로 인한 설정 누락과
환경 편차, 장애 복구 지연



GOAL

반복 가능한 인프라 · 배포 자동화와
보안 · 복구 가능한 운영 환경 구축



FUNCTION

IaC · CI/CD · WAF · RDS 복구 · 모니터링

01. PERIOD

수행 기간

2026.01.26 ~ 2026.03.05
6 WEEKS

02. TEAM / ROLE

참여 인원 · 본인 역할

5인 팀 프로젝트
인프라 자동화 · CI/CD 담당
- Terraform 기반 AWS 리소스 프로비저닝
- GitHub Actions - Ansible 자동 배포 구성
- K3s 및 애플리케이션 배포 연계

03. CONTRIBUTION

기여도

담당 영역 기여도 80%
IaC 구축 · CI/CD 파이프라인 · 배포 자동화 연계

04. STACK

사용 기술 · 도구

MY STACK

AWS · Terraform · Ansible
GitHub Actions · Docker · K3s

PROJECT STACK

RDS · NGINX WAF · Grafana · VictoriaMetrics

PROJECT SUMMARY

프로젝트 개요

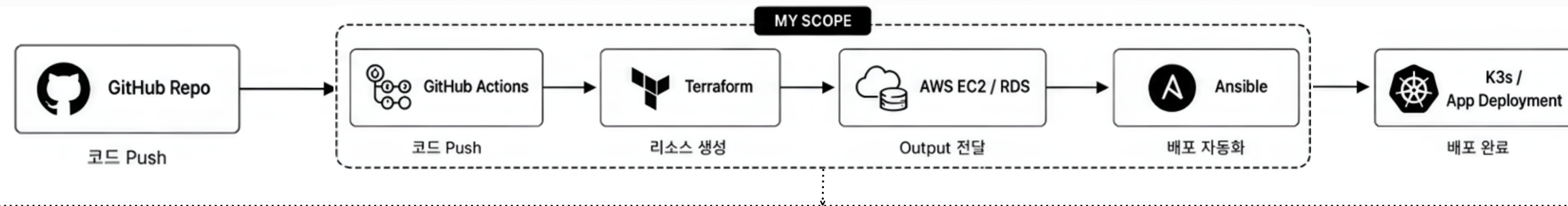
AWS 기반 인프라 구축부터 서비스 배포까지 자동화하고,
보안 · 백업 · 모니터링을 통합한 클라우드 운영 환경 구축 프로젝트입니다.
Terraform으로 AWS 리소스를 코드화하고, GitHub Actions와 Ansible을
연계해 K3s 환경과 애플리케이션 배포를 자동화했습니다. 또한 팀 프로젝트를
통해 WAF 보안, RDS 복구, 모니터링까지 확장했습니다.

PROJECT 1

역할 시각화 및 트러블 슈팅

02

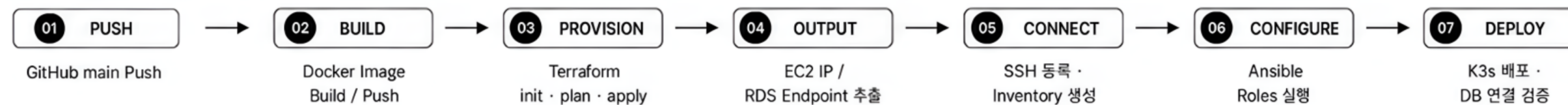
01. MY SCOPE ARCHITECTURE



TEAM-SCOPE SYSTEMS



02. AUTOMATION WORKFLOW



03. TROUBLESHOOTING

TROUBLE 01		보안 그룹 중복 생성 문제
PROBLEM	Terraform apply로 서버 배포 시 sg.tf가 함께 적용되며 보안 그룹이 매번 새로 생성되어 중복 증가	
SOLUTION	sg.tf 전용 브랜치를 별도로 생성해 보안 그룹을 초기 1회 생성하고, 이후 main 브랜치에서는 해당 리소스를 참조하도록 구조 설계	
OUTCOME	Terraform apply를 반복 수행해도 기존 ec2-sg 보안 그룹을 참조하여 중복 생성 방지	

TROUBLE 02		Terraform Output 연계 오류
PROBLEM	EC2 IP · RDS Endpoint 전달 실패로 Ansible 배포 중단	
CAUSE	Terraform Output과 Actions 변수 참조 불일치	
ACTION	outputs.tf 정비 및 terraform output -raw 적용	
RESULT	수동 입력 없는 연속 배포 구현	

PROJECT 1

성과 • 결과 및 직무 적용

03

01. VERIFIED OUTCOMES



DEPLOYMENT

선행 리소스 배포 →
MAIN 1 PUSH

Main 배포 4분 13초

GitHub Push 1회로 인프라 생성부터
K3s 애플리케이션 배포까지 자동 실행



SECURITY GROUP

반복 생성 → 중복 0건

초기 1회 생성 · 이후 기존 SG 참조

sg 전용 브랜치 분리 후 main에서
기존 ec2-sg를 참조하도록 개선



RDS RECOVERY

Snapshot 복구 자동화
→ 8분 28초

rds-dr 브랜치 기반 자동 복구

snapshot 복원부터 애플리케이션
DB 재연결까지 자동화



SECURITY TEST

200 OK / 403 Forbidden

정상 요청 허용 · 공격 패턴 차단

ModSecurity WAF 적용 후
정상 요청과 공격 요청을 검증



WORKFLOW RUNTIME

SG 27초

|

RDS 8분 55초

|

MAIN 4분 13초

|

DR 8분 28초

※ GitHub Actions 실행 로그 기준, AWS 실습 환경에서 측정한 결과

02. RESULTS



MY RESULT

- Terraform 기반 SG · RDS · EC2 프로비저닝 자동화
- GitHub Actions-Ansible-K3s 배포 파이프라인 구축
- 리소스 중복 생성 방지 및 Output 전달 구조 안정화



PROJECT RESULT

- 5개 Ansible Role과 8개 Pod 기반 운영 구성
- WAF 적용 후 정상 요청 200 / 공격 요청 403 검증
- RDS 자동 백업 및 Snapshot 기반 약 8분 복구 구현
- VictoriaMetrics · Loki · Grafana 기반 통합 모니터링 구성

03. RETROSPECTIVE & JOB APPLICATION



LEARNED

자동화의 핵심은 리소스의 생성 순서와 의존성, 상태를 관리해 동일한 결과를반복해서 재현하도록 설계하는 것임을 배웠습니다.



IMPROVEMENT

Free Tier 단일 노드 환경의 한계를 경험하며, 향후에는 Terraform Module · Remote State, Multi-AZ, 최소권한 IAM, SSH 접속, 모니터링 알림 구조로 확장하고자 합니다.



JOB APPLICATION

실무에서는 IaC · CI/CD · 보안 · 모니터링을 하나의 운영 흐름으로 설계하고, 실행 로그와 Output 기반의 장애 분석 및 복구 자동화를 통해 서비스 안정성을 높이는 클라우드 엔지니어로 기여하겠습니다.

PROJECT 2

01

하이브리드 클라우드 기반 중앙 집중형 에러 복구 파이프라인



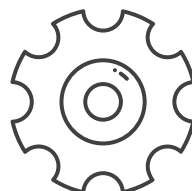
BACKGROUND

수작업 장애 대응으로 인한
복구 지연, 추가 사고 및 운영망 영향



GOAL

장애 감지부터 격리 · 재현 · 검증 · 복구까지
표준화된 자동 대응 체계 구축



FUNCTION

DLQ · Sandbox · Slack Approval ·
GitOps · Redrive

01. PERIOD

수행 기간

2026.03.10 ~ 2026.04.09
5 WEEKS

02. TEAM / ROLE

참여 인원 · 본인 역할

6인 팀 프로젝트
부팀장 · 백엔드 자동화 담당
- Kafka DLQ 처리 로직 구현
- 실패 원인 분석 및 Sandbox 명세 생성
- Slack 승인 · GitHub Actions 연동

03. CONTRIBUTION

기여도

담당 영역 기여도 90%
DLQ 처리 · Slack 승인 · 복구 자동화

04. STACK

사용 기술 · 도구

MY STACK

Python · FastAPI · Flask
Kafka · GitHub Actions · Slack · Argo CD

PROJECT STACK

AWS · K3s · Tailscale
Terraform · Ansible · Istio Ambient Mesh · RDS · Fluent bit
Prometheus · Grafana

PROJECT SUMMARY

프로젝트 개요

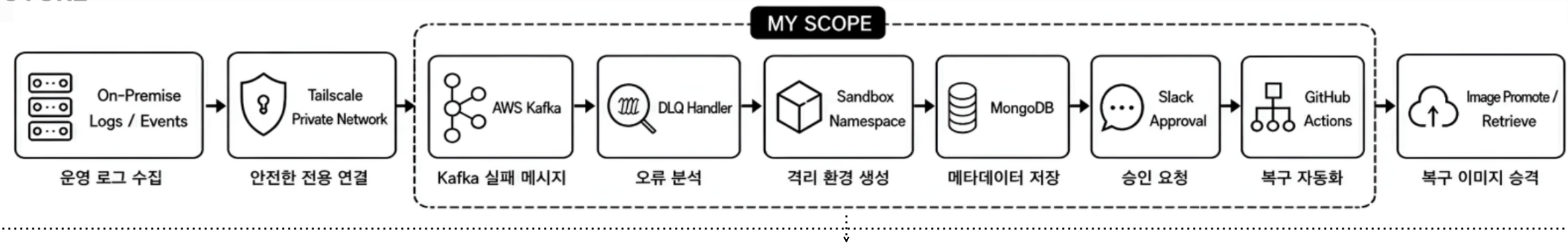
운영 메시지 처리 실패 시 오류 정보와 Payload를 Kafka DLQ로 격리하고,
동일한 장애를 재현할 수 있는 샌드박스를 자동 생성한 프로젝트입니다.
개발자는 격리 환경에서 수정 · 검증을 수행하고, 검증된 결과는 Slack 승인과
GitHub Actions를 통해 운영 이미지로 승격하여 실패 메시지를 재처리하도록
복구 파이프라인을 구성했습니다.

PROJECT 2

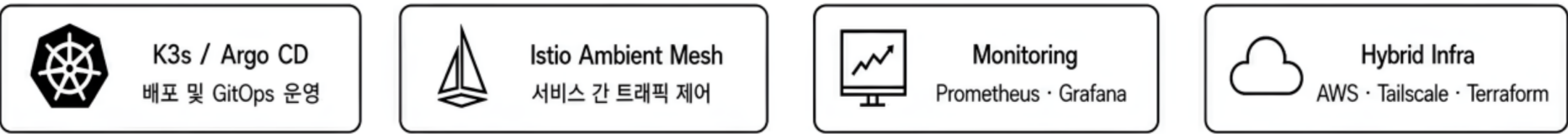
역할 시각화 및 트러블 슈팅

02

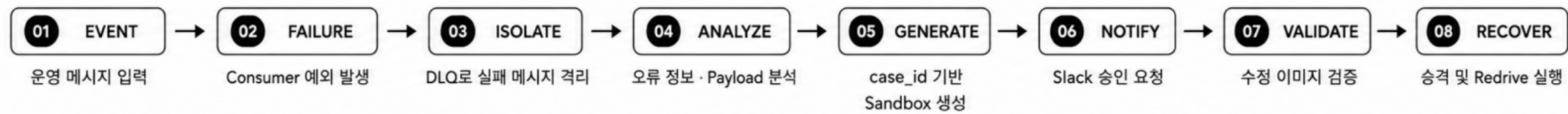
01. MY SCOPE ARCHITECTURE



TEAM-SCOPE SYSTEMS



02. AUTOMATION WORKFLOW



03. TROUBLESHOOTING

TROUBLE 01

다중 장애 시 샌드박스 충돌 문제

PROBLEM	동시 다발 장애 발생 시 Sandbox YAML과 상태 정보가 충돌해 사건 구분이 어려움
CAUSE	모든 장애가 공용 Sandbox 리소스와 공통 토픽 구조를 공유
SOLUTION	case_id 기반 Manifest · Deployment · 임시 DB · Kafka 결과 토픽과 상태 기록을 분리
OUTCOME	장애별 독립 실행과 상태 추적이 가능한 사건 단위 Sandbox 구조 구현

TROUBLE 02

Slack 승인 404 연동 오류

PROBLEM	Slack 승인 버튼 클릭 시 404 응답으로 후속 GitHub Actions 자동화 중단
CAUSE	Slack Callback URL과 FastAPI Endpoint 매핑 불일치 및 권한 설정 누락
SOLUTION	Slack 전용 Endpoint를 추가하고 Callback URL · 권한 · Payload 처리를 재정비
OUTCOME	Slack 승인부터 GitHub Actions 실행까지 양방향 연동 및 복구 흐름 정상화

PROJECT 2

성과 · 결과 및 직무 적용

03

01. VERIFIED OUTCOMES

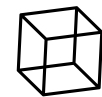


CASE GENERATION

장애 이벤트 →
case_id 생성

Failure · Normal · Redrive Artifact 분리

DLQ 메시지의 Payload와 오류 정보를 기준으로
사건별 입력 데이터와 상태 기록 생성



CASE ISOLATION

공용 환경 →
1 CASE : 1 SANDBOX

공용 Namespace 내 case_id별 Sandbox
리소스 생성

Manifest · Deployment · 임시 DB · Topic ·
Case Record 분리

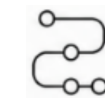


CHATOPS APPROVAL

수동 명령 → 1 CLICK

Slack 승인 후 GitHub Actions 실행

운영자가 CLI로 직접 조회하지 않고
Slack 버튼으로 후속 복구 Workflow 승인



RECOVERY PIPELINE

실패 로그 → 8 STAGES

장애 수집 · 재현 · 검증 · 승격 · 재처리

메시지 실패부터 재현 · 검증 · 이미지 승격 ·
Kafka 재처리까지 여러 GitHub Actions
Workflow로 자동 연결



RECOVERY VALIDATION

CASE GENERATION

CASE ISOLATION

CHATOPS 1 CLICK

RECOVERY 8 STAGES

※ 개발 테스트 로그와 시연 시나리오를 기준으로 검증한 결과

02. RESULTS



MY RESULT

- Kafka DLQ 기반 실패 메시지 격리 및 Artifact 생성
- Merlion 기반 이상 탐지 및 Slack 알림 시나리오 검증
- case_id 기반 Sandbox 명세와 Case Record 생성
- Slack 승인-GitHub Actions 연동 및 Redrive 복구 흐름 구축



PROJECT RESULT

- Tailscale 기반 On-Premise-AWS 사설 네트워크 통합
- Terraform · ArgoCD 연계 분리형 GitOps 구현
- NetworkPolicy · Istio 기반 Sandbox 격리
- Prometheus · Grafana 기반 Pod · 서비스 모니터링 구현

03. RETROSPECTIVE & JOB APPLICATION



LEARNED

실패 데이터 보존 → 격리 → 재현 → 검증 → 승인 → 재처리가 하나의 운영 통제 과정임을 배웠습니다.
Merlion 단독 판단의 한계를 baseline과 persistence로 보완하며,
AI 모델과 운영 규칙을 함께 설계하는 중요성을 체감했습니다.



IMPROVEMENT

Namespace 전체 보안 정책, Multi-Region DR,
운영 자원 확대 및 Audit Log 기반 검증 구조로
확장하고자 합니다.



JOB APPLICATION

Kafka DLQ · ChatOps · GitOps 기반 장애 대응을 표준화하고,
격리 환경에서 검증된 결과만 운영에 반영하겠습니다.
로그 · 모델 · 운영 규칙을 함께 활용해 오탐과 장애 확산을 줄이는
클라우드 운영 자동화 엔지니어로 기여하겠습니다.

PROJECT 3

01

AWS 단일 클라우드 기반 AIOps · FinOps · DR 통합 운영 플랫폼



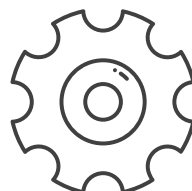
BACKGROUND

운영 데이터 분산, 사후 대응 중심 운영,
비용 · 성능 · DR 체계 분리



GOAL

관측 · 분석 · 판단 · 대응 흐름을 통합하고,
운영 · 보안 · DR 체계를 자동화



FUNCTION

AIOps · FinOps · DR · CI/CD · Security

01. PERIOD

수행 기간

2026.04.16 ~ 2026.07.21
14 WEEKS

02. TEAM / ROLE

참여 인원 · 본인 역할

5인 팀 프로젝트
PM · CI/CD · Security 담당
- GitHub Actions 기반 인프라 자동화 및 배포 파이프라인 구축
- OIDC · IAM · KMS · Secrets Manager · ESO 기반 보안 체계 구현
- 팀별 기능 일정 관리 및 통합 검증 시나리오 조율

03. CONTRIBUTION

기여도

담당 영역 기여도 95%
CI/CD · Security 설계 및 구현 주도

04. STACK

사용 기술 · 도구

MY STACK

AWS · Terraform · Ansible · GitHub Actions ·
KMS · Secrets Manager · ESO · Trivy · Checkov

PROJECT STACK

Kubernetes · Cilium · Prometheus · Grafana ·
Kafka · PostgreSQL · Discord

PROJECT SUMMARY

프로젝트 개요

AWS 환경의 운영 메트릭과 비용 데이터를 통합 수집·분석하고, AIOps·FinOps·DR을 하나의 운영 흐름으로 연결한 플랫폼입니다. Terraform·Ansible·GitHub Actions로 인프라와 Kubernetes 구성을 자동화했으며, OIDC·KMS·Secrets Manager·ESO와 보안 스캔을 적용해 배포 전후 보안 체계를 구축했습니다. 또한 서울 Primary와 오사카 DR 환경을 구성해 운영 안정성과 복구 대응력을 강화했습니다.

PROJECT 3

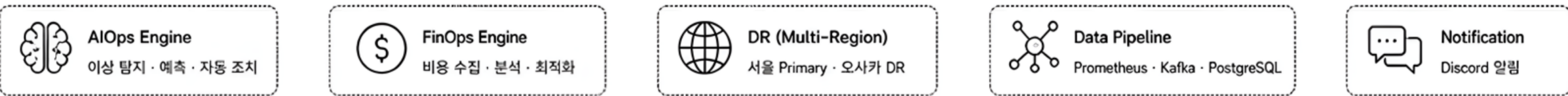
역할 시각화 및 트러블 슈팅

02

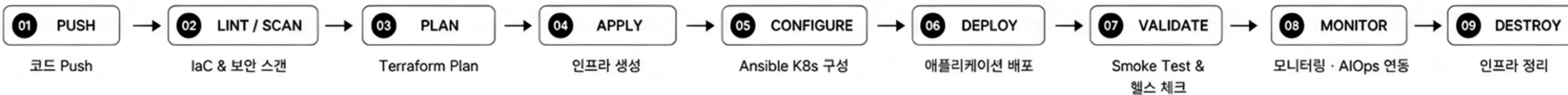
01. MY SCOPE ARCHITECTURE



TEAM-SCOPE SYSTEMS



02. AUTOMATION WORKFLOW



03. TROUBLESHOOTING

TROUBLE 01 NetworkPolicy 적용 후 ALB Health Check 실패	
PROBLEM	NetworkPolicy 적용 이후 ALB Target이 unhealthy 상태로 유지
CAUSE	Cross-node Pod 통신에 필요한 Pod CIDR 대역이 차단되어 헬스 체크 실패
SOLUTION	NetworkPolicy에 Pod 네트워크 대역(192.168.0.0/16) 추가 및 Deployment Rollout Restart
OUTCOME	등록된 ALB Target 전체 Healthy 복구, 보안 정책 유지하며 서비스 정상화

TROUBLE 02 GitHub Actions SSH 접근 및 보안 강화	
PROBLEM	GitHub Runner IP가 매번 변경되어 SSH 접근 및 보안 관리 어려움
CAUSE	고정 IP 부재로 인해 SSH 22번 포트를 상시 열어둘 수 없는 구조
SOLUTION	Runner Public IP 조회 → /32 임시 허용 → 작업 수행 → 규칙 자동 회수
OUTCOME	자동화 파이프라인 보안 유지 및 안정적인 배포 구현

PROJECT 3

성과 · 결과 및 직무 적용

03

01. VERIFIED OUTCOMES



CI/CD VALIDATION

4 Nodes Ready

Control Plane 1 · Worker 3

Provision 완료 후 kubectl wait 기반 검증
Kubernetes 노드 Ready 상태 확인



LIFECYCLE WORKFLOW

2 Workflows

Provision · Destroy

동일 Remote State 기반 인프라 생성 · 정리
E2E 라이프사이클 자동화 검증



SECURITY VALIDATION

OIDC · Hard Gate

Checkov · Trivy

장기 Access Key 제거 및 OIDC 기반권한 적용
정적 Security Gate로 배포 전 보안 검증



SERVICE HEALTH

ALL TARGETS HEALTHY

Smoke Test 통과

NetworkPolicy 보완 후 등록된 ALB Target
전체의 정상 상태 확인



VALIDATION HIGHLIGHTS

NODE READY 4

WORKFLOWS 2

ALB TARGETS HEALTHY

SECURITY GATE PASS

※ GitHub Actions · Kubernetes · AWS 실환경 기준 검증 결과

02. RESULTS



MY RESULT

- GitHub Actions 기반 Terraform · Ansible 자동화 파이프라인 구축
- OIDC · IAM · KMS · Secrets Manager · ESO 보안 체계 구현
- Checkov · Trivy Hard Gate 및 SSH /32 임시 허용 자동화 적용
- ALB Health Check · Smoke Test 기반 배포 검증 안정화



PROJECT RESULT

- AWS 단일 클라우드 기반 AIOps · FinOps · DR 통합 운영 플랫폼 구현
- Prometheus · Kafka · PostgreSQL 기반 운영 파이프라인 구축
- 서울 Primary · 오사카 DR 및 PostgreSQL Cross-Region Replica 구성
- Discord 연동 AIOps 자동 조치 및 FinOps 비용 권고 체계 구현

03. RETROSPECTIVE & JOB APPLICATION



LEARNED

인프라 자동화는 Provision 성공만으로 끝나지 않고 Node Ready, Smoke Test, ALB Health Check까지 검증해야 완성된다는 점을 배웠습니다. 또한 OIDC, Secret 관리, 정적 보안 검사를 배포 파이프라인에 통합해 운영 안정성을 높였습니다.



IMPROVEMENT

향후에는 Multi-AZ Kubernetes, Route 53 Health Check 기반 DR 자동 전환, SSM Session Manager 기반 Private Access, AIOps 조치의 Canary · Rollback 정책까지 확장하고자 합니다.



JOB APPLICATION

이 경험을 통해 IaC, Kubernetes, 보안, 모니터링, DR을 하나의 운영 체계로 연결하는 역량을 쌓았습니다. 실서비스 검증과 로그 · 네트워크 기반 문제 해결 경험을 바탕으로 클라우드 인프라 구축 · 운영 자동화 직무에 기여하겠습니다.

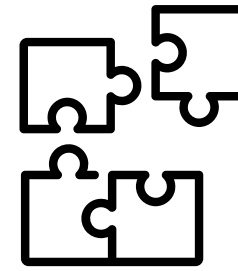
CORE COMPETENCIES



운영 안정성을 고려한 인프라 설계

Terraform으로 AWS 인프라를 자동화하는 과정에서 보안 그룹 중복 생성과 Output 전달 오류를 발견했습니다. 리소스 생성 기준과 변수 참조 구조를 개선하고, 배포 이후 WAF 요청 검증과 RDS 복구까지 확인해 반복 실행이 가능한 안정적인 인프라 환경을 구축했습니다.

#리소스 검증 #재현 가능한 인프라 #운영 안정성



로그 기반 문제 해결 능력

Kafka DLQ에 적재된 실패 메시지를 분석하고, 장애별 case_id를 기준으로 Sandbox를 분리했습니다. 장애 발생 시 격리 환경이 충돌하는 문제와 Slack 승인 연동 오류를 로그 · Callback URL · 메타데이터 흐름 단위로 추적해, 장애 격리부터 승인 · 검증 · 재처리까지 8단계 복구 파이프라인으로 구조화했습니다.

#장애 분석 #원인 추적 #복구 프로세스 설계

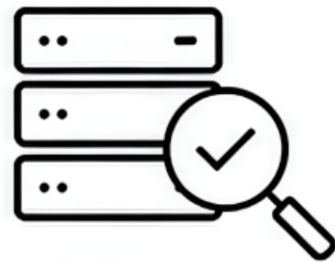


보안을 내재화한 통합 자동화 역량

PM과 CI/CD · Security 담당으로 인프라 생성부터 Kubernetes 구성, 보안 검사, 배포 검증, 인프라 정리까지 End-to-End Workflow를 구축했습니다. OIDC · 최소 권한 IAM · KMS · Secrets Manager · ESO · Checkov · Trivy를 배포 과정에 연결하고, 팀별 AIOps · FinOps · DR 기능의 통합 검증을 조율했습니다.

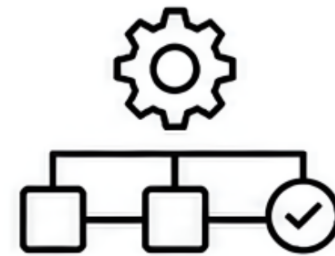
#DevSecOps #E2E 자동화 #기능 통합

입사 후 포부



빠르게 이해하고 안정적으로 운영하겠습니다

입사 초기에는 서비스 아키텍처와 배포 구조, 운영 정책을 빠르게 파악하겠습니다. 모니터링 지표와 장애 이력, Runbook을 기반으로 시스템의 정상 상태를 이해하고, 작은 변경도 단계별 검증을 거쳐 서비스 안정성을 우선하는 엔지니어가 되겠습니다.



반복 업무를 표준화하고 자동화하겠습니다

Terraform · Ansible · CI/CD 경험을 활용해 반복되는 인프라 구축과 배포 작업을 코드와 Workflow로 표준화하겠습니다. 보안 검사, 배포 검증, 모니터링을 파이프라인에 연결해 작업 편차와 휴먼 에러를 줄이고, 장애 발생 시 원인을 빠르게 추적할 수 있는 운영 환경을 만들겠습니다.



데이터 기반 운영 체계를 고도화하겠습니다

장기적으로는 성능 · 비용 · 보안 · 가용성을 함께 판단할 수 있는 운영 체계를 구축하겠습니다. AIOps · FinOps · DR 경험을 바탕으로 이상 징후를 선제적으로 탐지하고, 비용 효율성과 복구 가능성까지 고려하는 클라우드 인프라 엔지니어로 성장하겠습니다.

구축 이후의 운영까지 책임지며, 안정적인 서비스를 만드는 클라우드 인프라 엔지니어가 되겠습니다.

THANK YOU

CONTACT

PHONE : 010 5628 7676 | MAIL : HASEOK612@NAVER.COM

HYPERLINK

[HTTPS://GITHUB.COM/HASEOLC?TAB=REPOSITORIES](https://github.com/HASEOLC?tab=repositories)